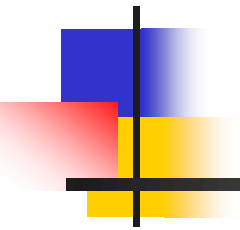
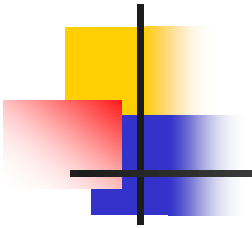


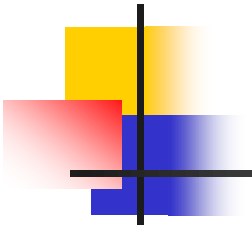
Chapter 8

Searching & Hashing

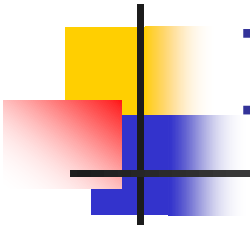


OBJECTIVE

- Introduces:
  - ✓ Basic searching concept
  - ✓ Type of searching
  - ✓ Hash function
  - ✓ Collision problems

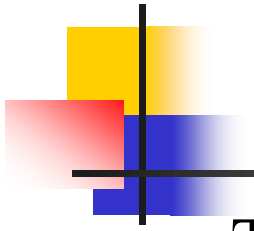


CONTENTS

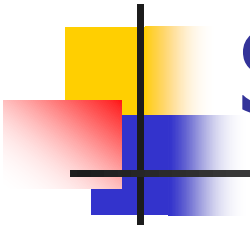


Introduction

- The most important operation on a list is the search algorithm. Using the search algorithm, we can:
  - Determine whether a particular item is in the list.
  - If the data is specifically organized (eg, sorted), find the location in the list where a new item can be inserted.
  - Find the location of the item to be deleted.



- There are so many techniques for searching, but none is the best because searching approach will rely on:
  - ✓ Speed and Space – choose of fast technique but wasted space is not a good idea (slower technique with optimized space is much better!)
  - ✓ Static and dynamic tables – complexity of table must take into account (they need to be considered!)
  - ✓ Table size – time (short or long) taken to search a data is depending on the size of table.

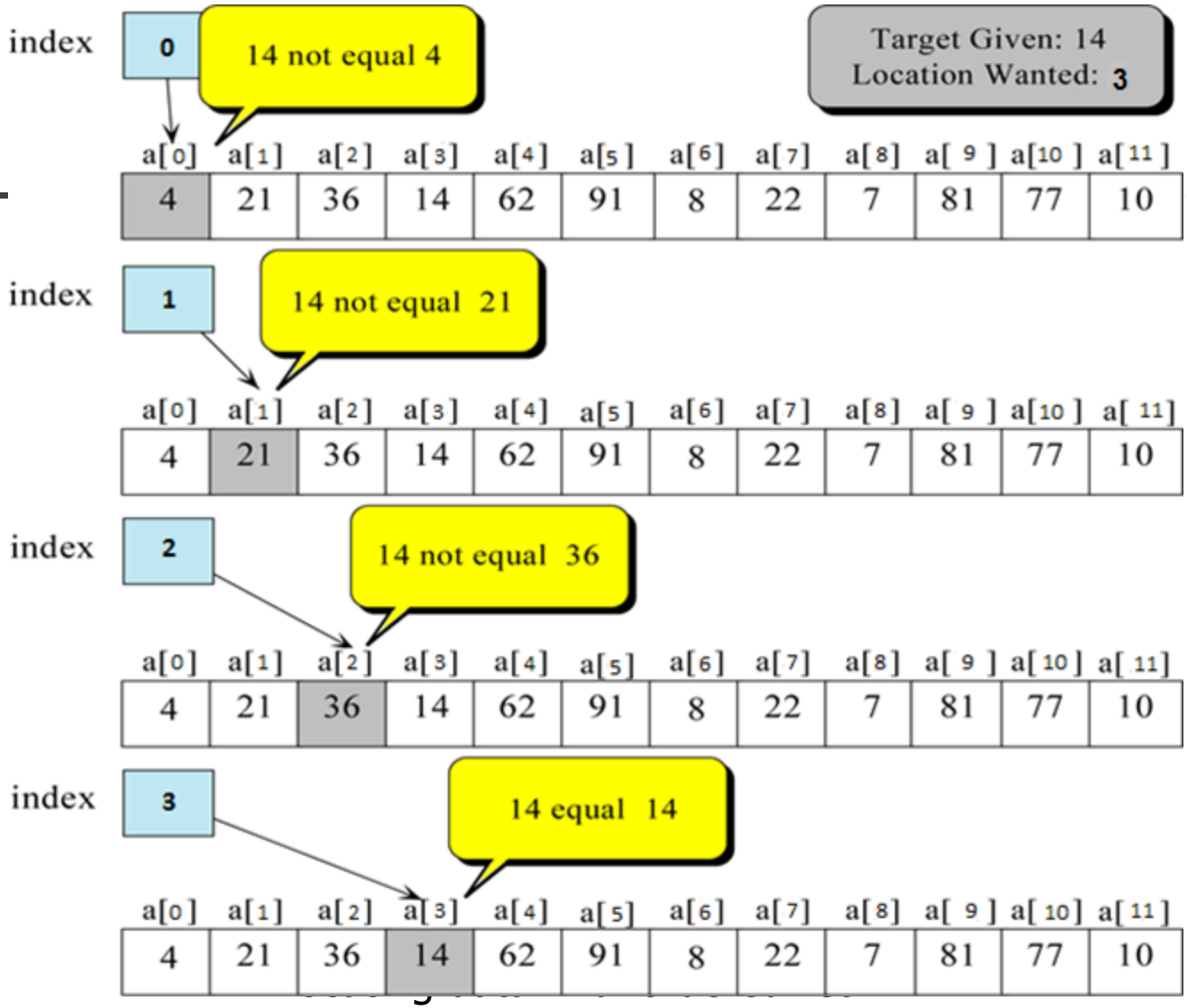


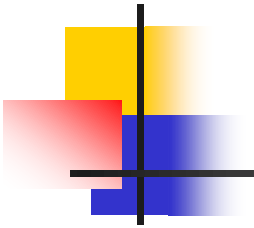
Searching algorithms

- Searching approaches that can be used:
  - ✓ Sequential (Linear) search
  - ✓ Binary search
  - ✓ binary trees
  - ✓ AVL Trees
  - ✓ B Trees
  - ✓ Hashing

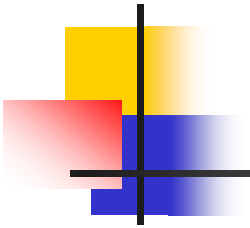
8.2.1 Sequential Search

- Search an *array or list by checking items one at a time.*
- ***Sequential* search** is usually very simple to implement, and is practical when the list has only a few elements, or when performing a single search in an unordered list.
- **Look at every element :** This is a very straightforward loop comparing every element in the array with the target(key).
  - Eighter we find it,
  - or we reach the end of the list!


index
0
14 not equal 4
Target Given: 14
Location Wanted: 3
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 21 36 14 62 91 8 22 7 81 77 10
index
1
14 not equal 21
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 21 36 14 62 91 8 22 7 81 77 10
index
2
14 not equal 36
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 21 36 14 62 91 8 22 7 81 77 10
index
3
14 equal 14
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 21 36 14 62 91 8 22 7 81 77 10



- The searching algorithm requires three parameters:
  1. The list.
  2. An index to the last element in the list.
  3. The target.



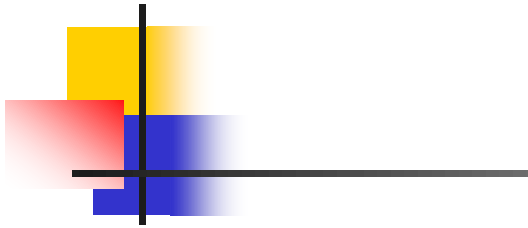
algorithm SeqSearch (val list <array>, val last <index>,
val target <keyType>)

Locate the target in an unordered list of size elements.

PRE list must contain at least one element.
last is index to last element in the list.
target contains the data to be located.

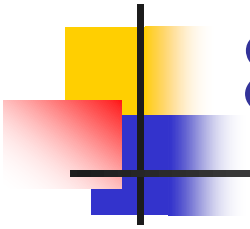
POST if found – matching index stored in Location.
if not found – (-1) stored in Location

RETURN Location<integer>



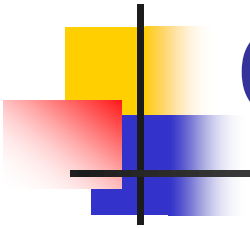
```
looker = 0
loop (looker < last AND target  not equal list(looker))
      looker = looker + 1
if (target  == list[looker] )
          Location= looker

else
          Location = -1
End If
return Location
end SeqSearch
```



8.2.2 Binary Search Algorithm

- Binary search algorithm is efficient if the array is sorted.
- A binary search is used whenever the list starts to become large.
- Consider to use binary searches whenever the list contains more than 16 elements.
- The binary search starts by testing the data in the element at the middle of the array to determine if the target is in the first or second half of the list.
- If it is in the first half, we do not need to check the second half. If it is in the second half, we do not need to test the first half. In other words we eliminate half the list from further consideration. We repeat this process until we find the target or determine that it is not in the list.



Cont'd

- To find the middle of the list, we need three variables, one to identify the beginning of the list, one to identify the middle of the list, and one to identify the end of the list.
- We analyze two cases here: the target is in the list (target found) and the target is not in the list (target not found).

8.2.2 Binary Search Algorithm

- **Target found case**: Assume we want to find 22 in a sorted list as follows:

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]	a[11]
4	7	8	10	14	21	22	36	62	77	81	91

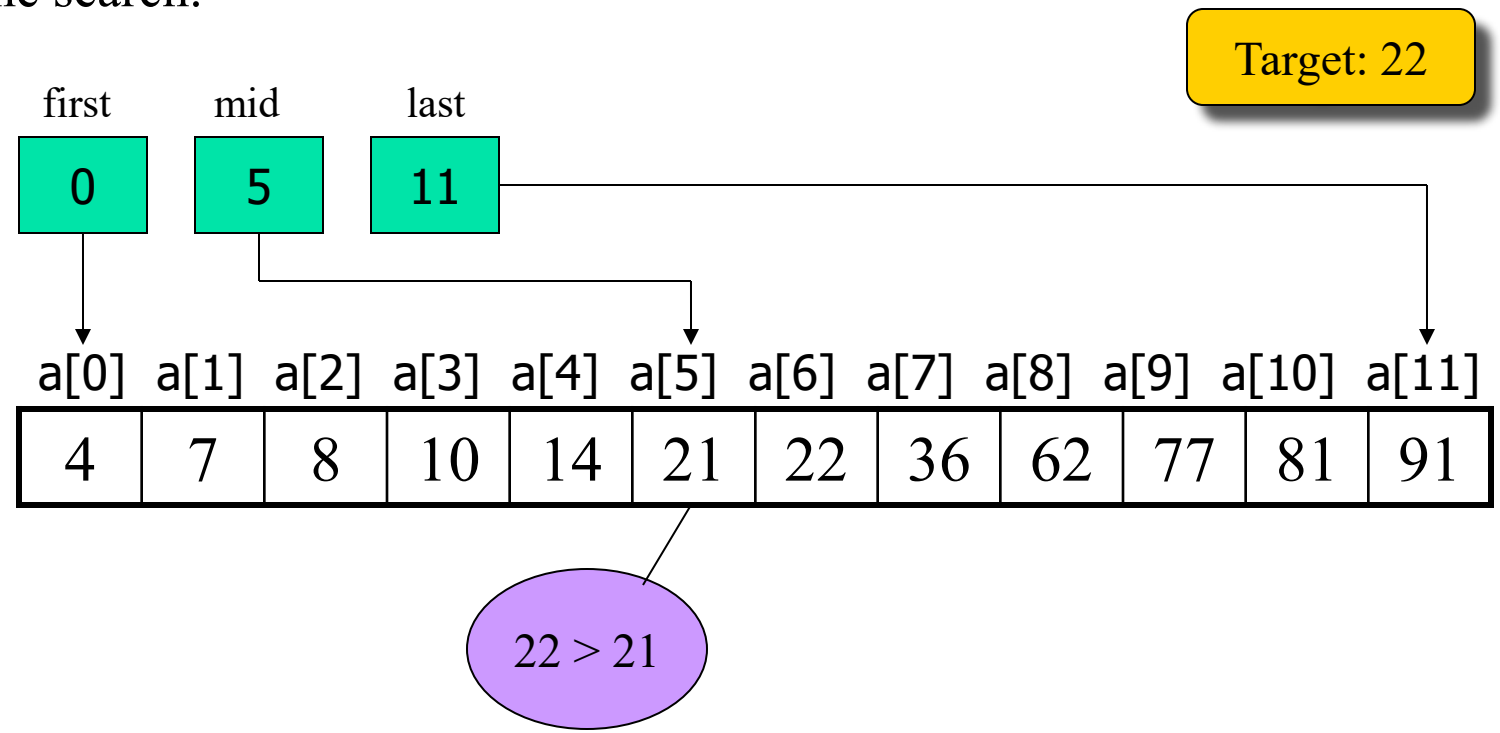
- The three indexes are first, mid and last. Given first as 0 and last as 11, mid is calculated as follows:

  mid = (first + last) / 2

  mid = (0 + 11) / 2 = 11 / 2 = 5

8.2.2 Binary Search Algorithm

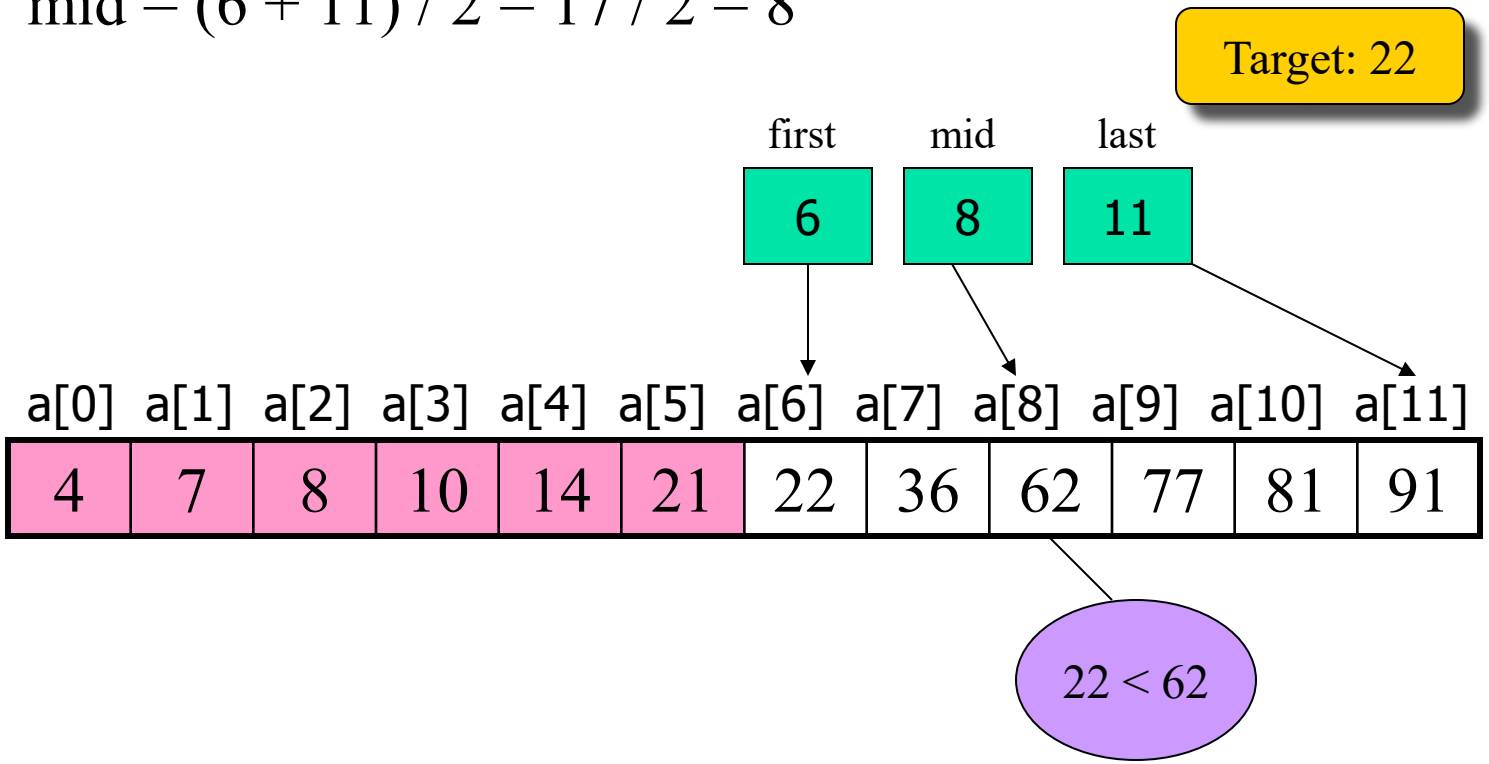
- At index location 5, the target is greater than the list value (22 > 21). Therefore, eliminate the array locations 0 through 5 (mid is automatically eliminated). To narrow our search, we assign mid + 1 to first and repeat the search.



8.2.2 Binary Search Algorithm

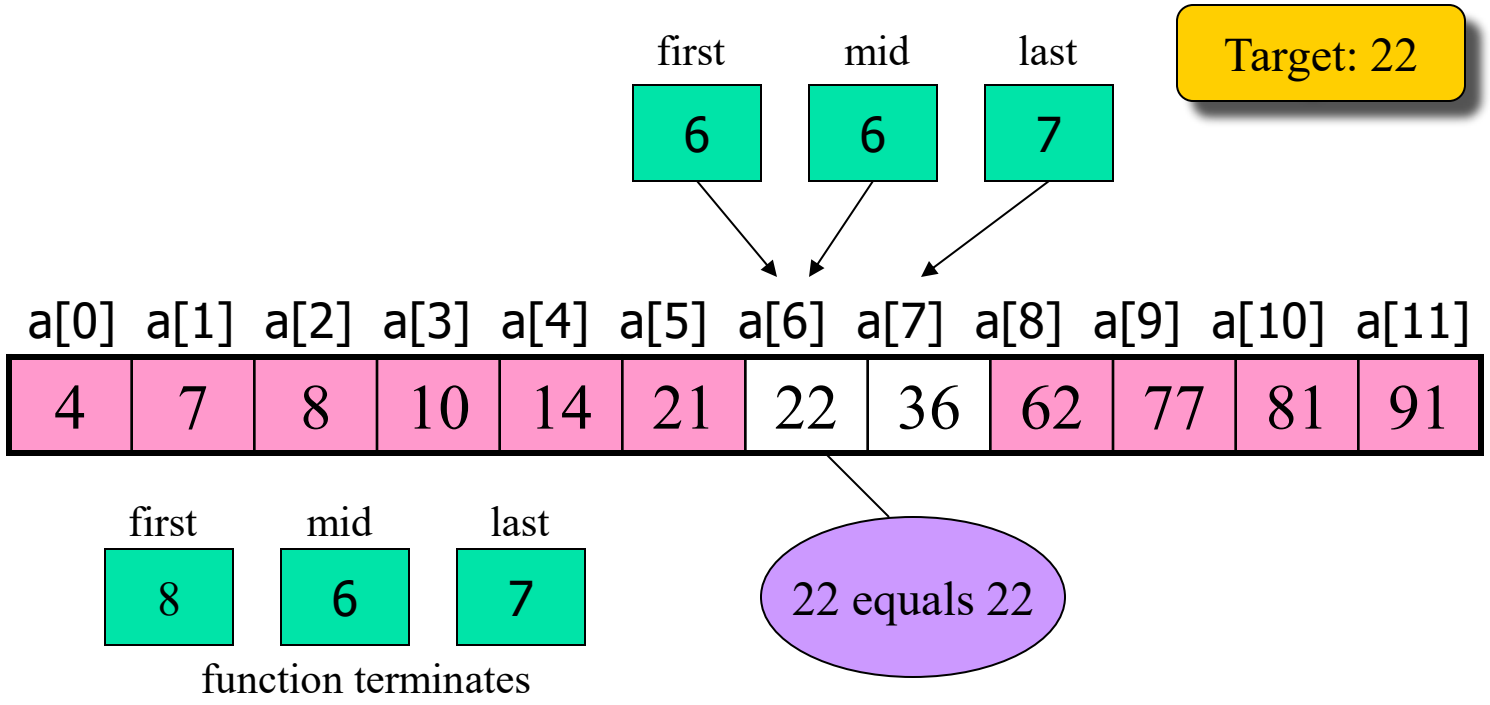
- The next loop calculates mid with the new value for first and determines that the midpoint is now 8 as follows:

  mid = (6 + 11) / 2 = 17 / 2 = 8



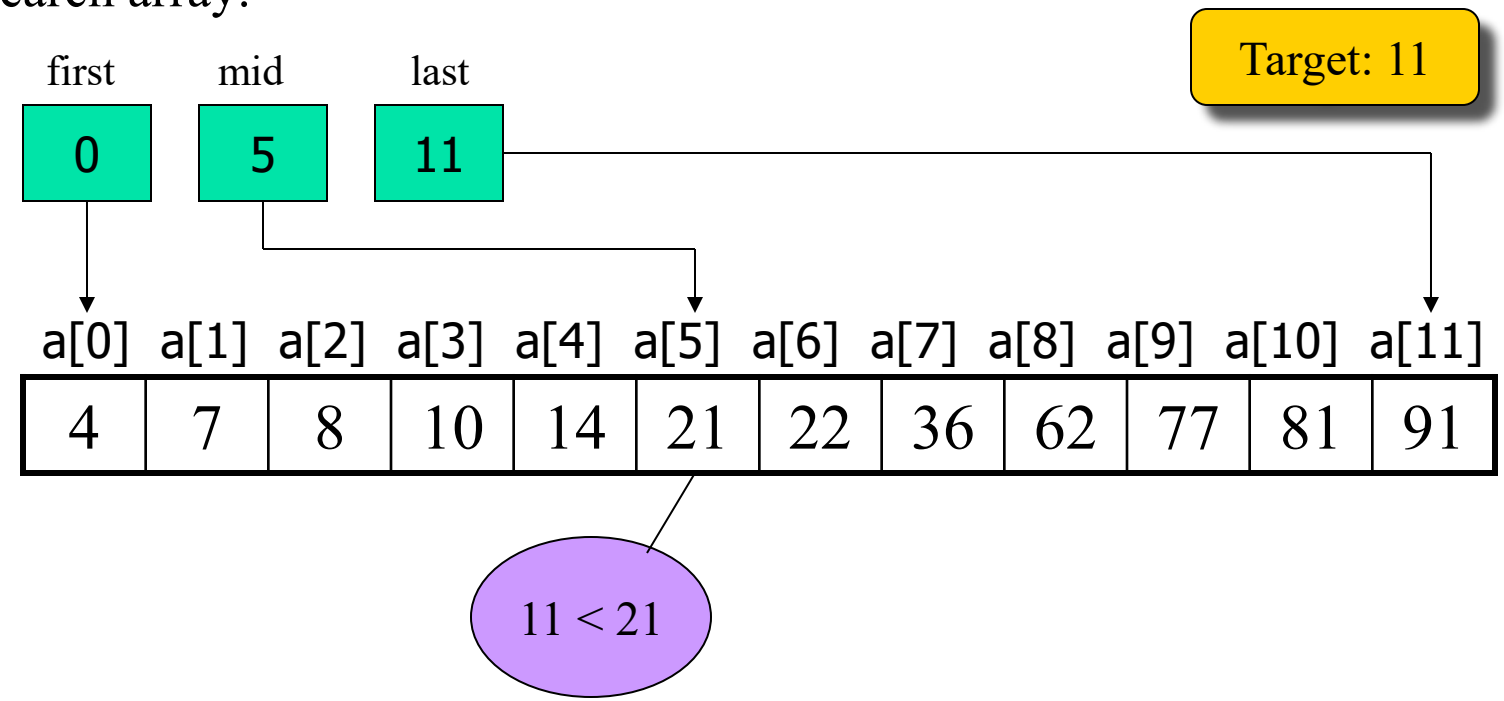
8.2.2 Binary Search Algorithm

- When we test the target to the value at mid a second time, we discover that the target is less than the list value (22 < 62). This time we adjust the end of the list by setting last to mid – 1 and recalculate mid. This step effectively eliminates elements 8 through 11 from consideration. We have now arrived at index location 6, whose value matches our target. This stops the search.



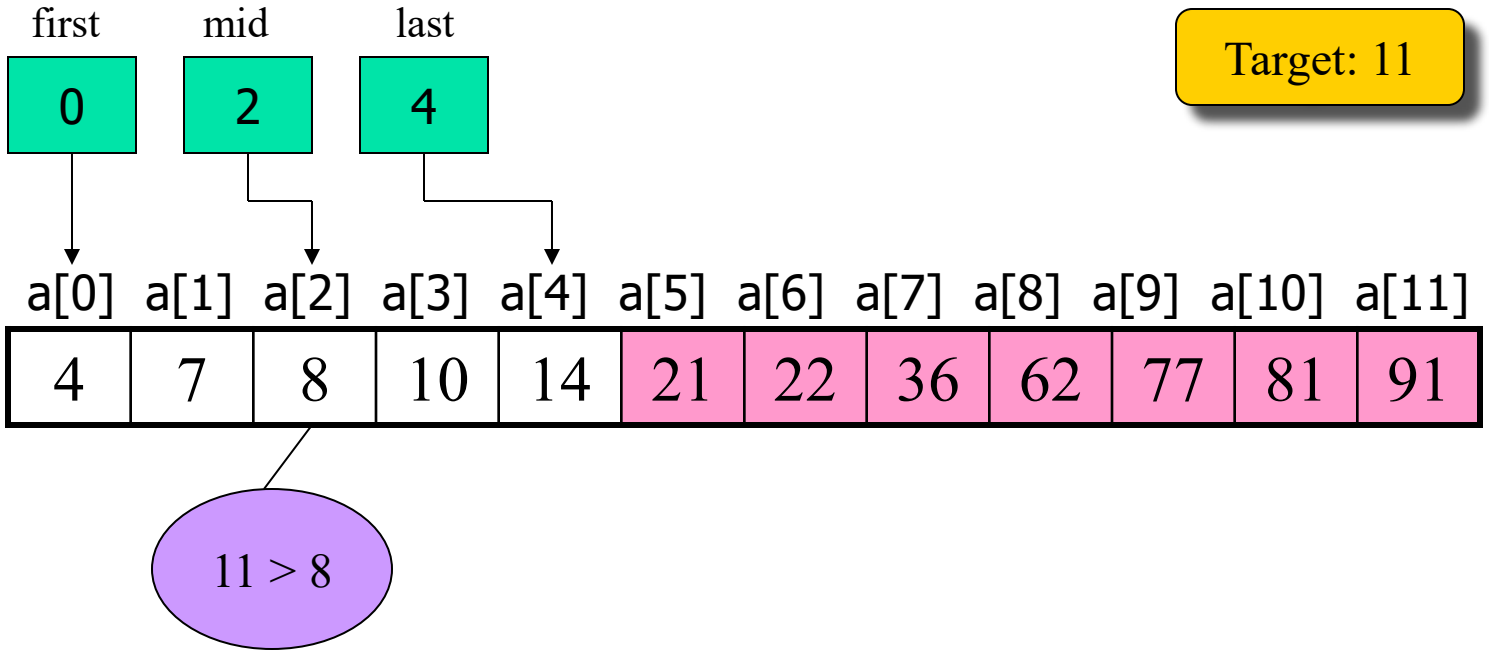
8.2.2 Binary Search Algorithm

- Target not found case: This is done by testing for first and last crossing: that is, we are done when first becomes greater than last. Two conditions terminate the binary search algorithm when (a) the target is found or (b) first becomes larger than last. Assume we want to find 11 in our binary search array.



8.2.2 Binary Search Algorithm

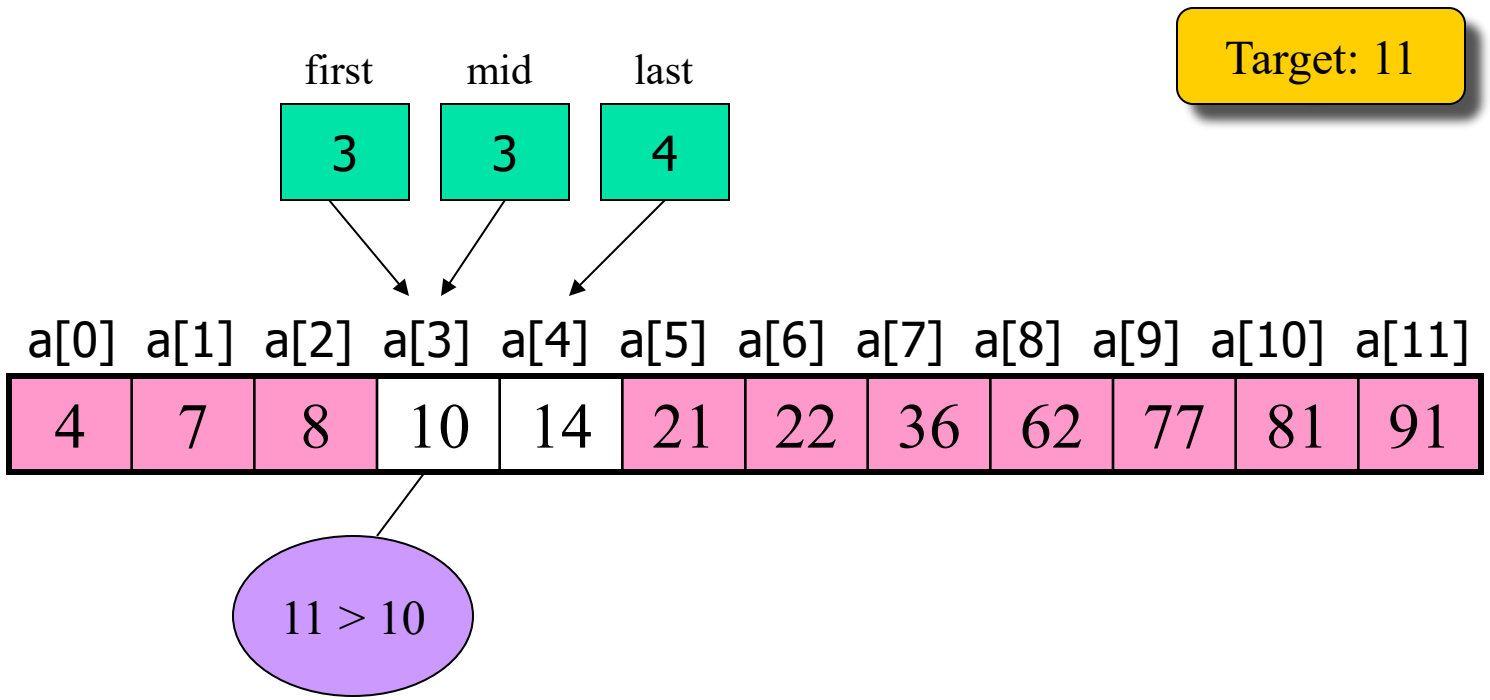
- The loop continues to narrow the range as we saw in the successful search until we are examining the data at index locations 3 and 4.



8.2.2 Binary Search Algorithm

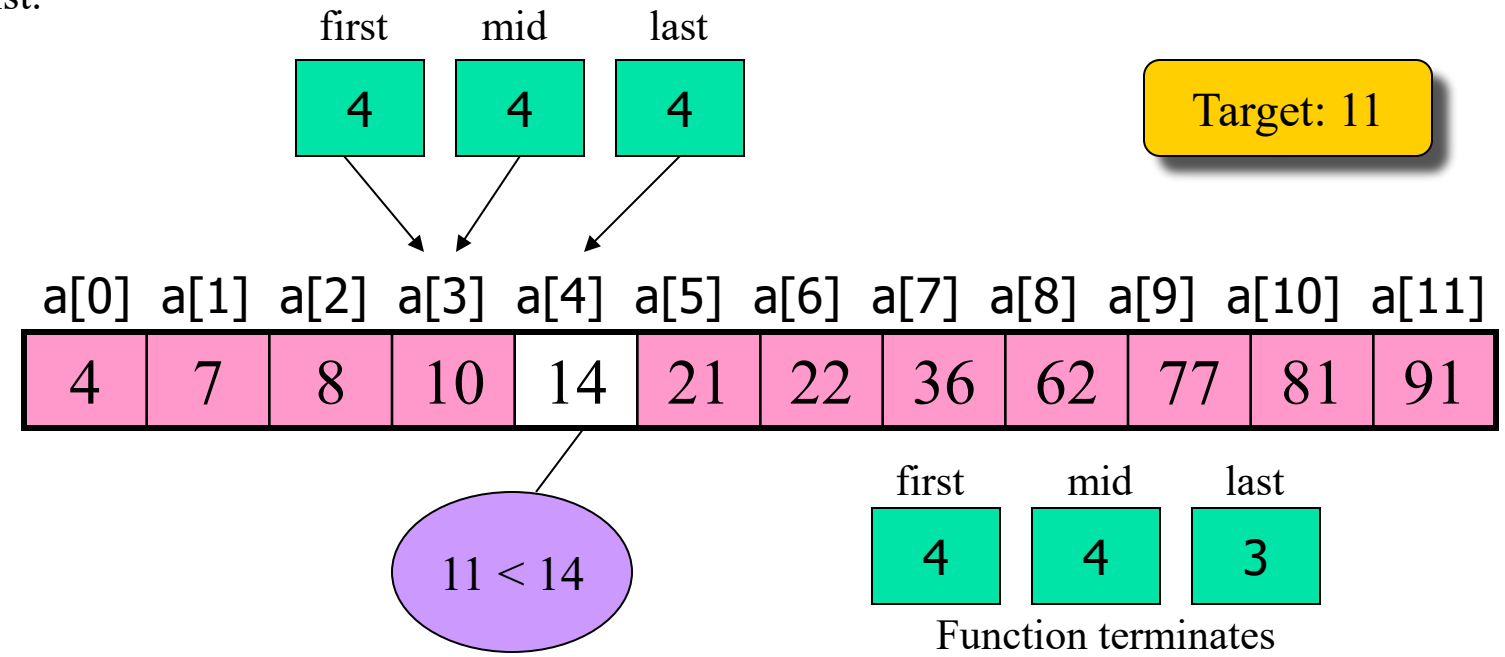
- These settings of first and last set the mid index to 3 as follows:

  mid = (3 + 4) / 2 = 7 / 2 = 3



8.2.2 Binary Search Algorithm

- The test at index 3indicates that the target is greater than the list value, so we set first to mid + 1, or 4. We now test the data at location 4 and discover that 11 < 14. The mid is as calculated as follows:
- At this point, we have discovered that the target should be between two adjacent values; in other words, it is not in the list. We see this algorithmically because last is set to mid – 1, which makes first greater than last, the signal that the value we are looking for is not in the list.



8.2.2 Binary Search Algorithm

- Example algorithm:

  DATA – sorted array

  ITEM – Info

  LB – lower bound

  UB – upper bound

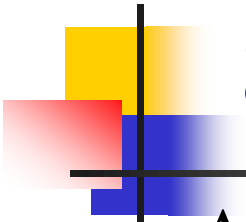
  ST – start Location

  MID – middle Location

  LAST – last Location

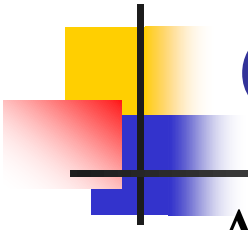
8.2.2 Binary Search Algorithm

1. [Define variables]
   ST = LB, LAST= UB;
   MID = (ST+LAST)/2;
2. Repeat 3 and 4 DO ST <= LAST & DATA[MID] != ITEM
3. If ITEM < DATA[MID] then
   LAST = MID-1
   If not
   ST = MID+1
4. Set MID = INT((ST + LAST)/2)
   [LAST repeat to 2]
5. If DATA[MID] = ITEM then
   LOK = MID
   If not,
   LOK = NULL
6. Stop



8.2 Hashing

- A hash search is a search in which the key, through an algorithmic function, determines the location of the data (in a defined table).
- Table (hash table) is a place for the data; that is for each entry it will keep a unique key value.
- Table entries will have their own unique key that is related to the data that has been entered to the table.
- Therefore, search is an operation that will use defined key in order to access data or information from the table.



Cont'd

- At the beginning of this chapter, searching technique is done by comparing each of data-key.
- Binary-search technique can provide a good performance for searching data if and only all the keys are in sorted sequence. But it will take time!!!
- Hashing is a search technique that requires keys in unsorted sequence and search by using the address index of the key.
- In this technique, data-storing process will also using hashing concept, that is hash index @ address index (address for particular information). This will requires hash function.

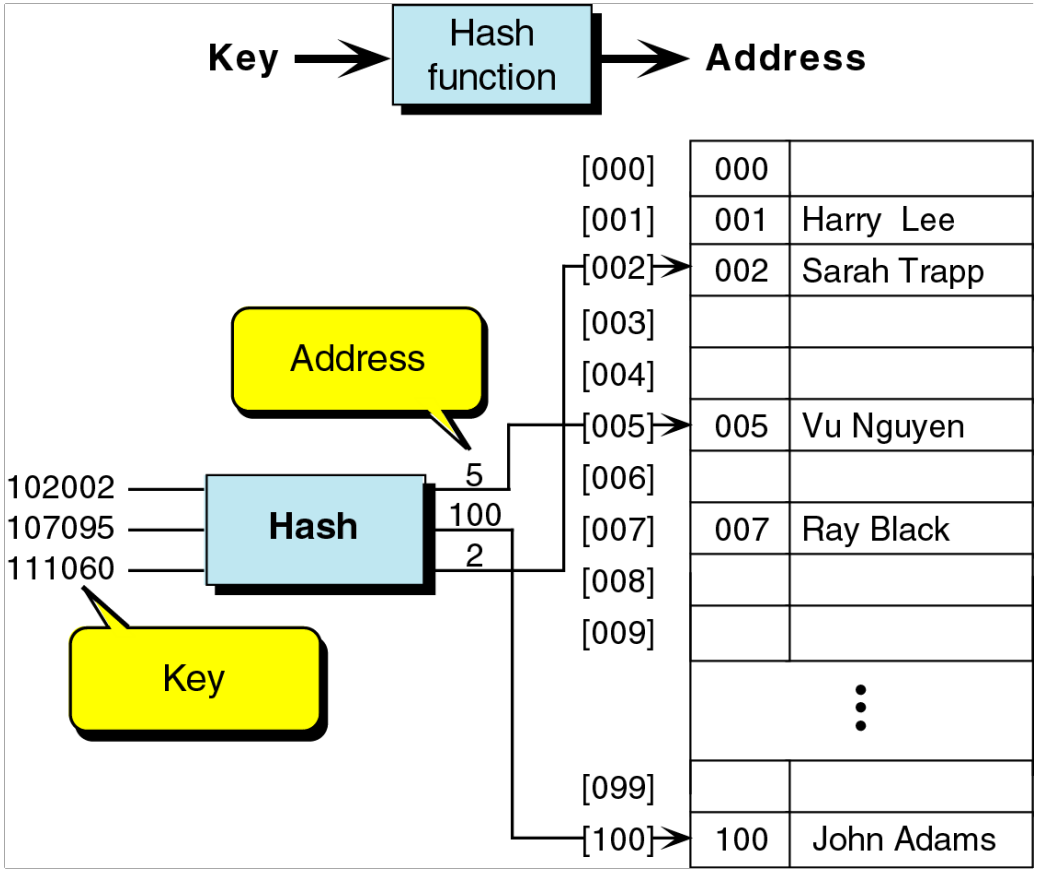
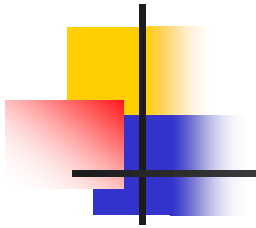


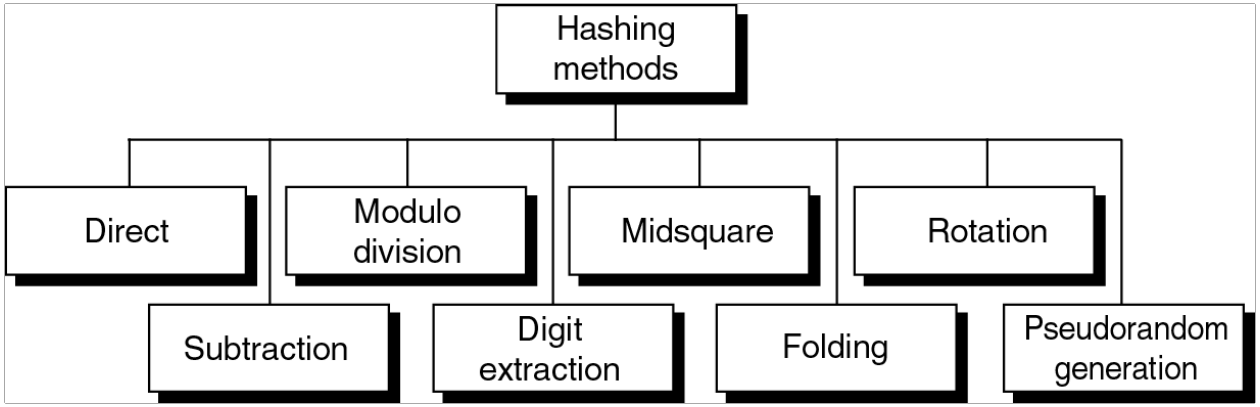
Fig. 1: Hashing concept

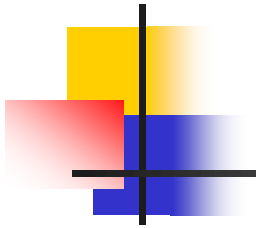


- The process of accessing all the data or information from hash table will use hash function to get the hash index.

8.2.1 Hash Function: Modul O-Division

- The selection of hash function to be used is very important because it will define the addressing approach for keys into the hash table.
- We are required to spread those keys into the hash table fair enough so that we can minimize the use of same address location (collision).





- Modulo-division is one of the hashing techniques that apply divide operation to find the address; it divides the key by the array/table size and uses the remainder for the address.

```
Address = key MOD listsize
```

Assume we have function

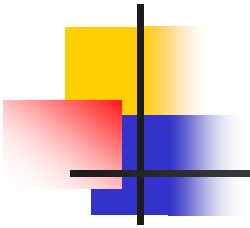
H(K) = K mod M; where:

K – key value

H – hash function

M – size of list / array / table

Address that is generated from H(K) : 0 < H(K) < M

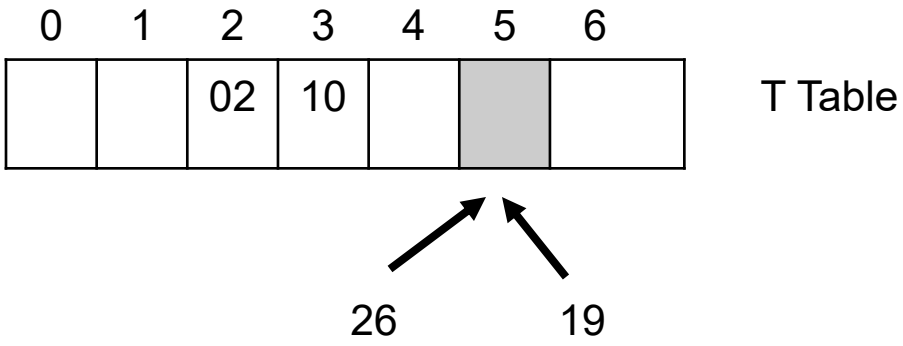


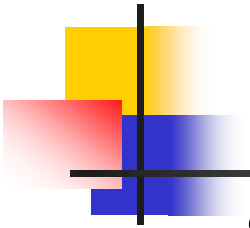
Collision

- Collision is a situation where two or more keys are pointed to the same address location (this normally happened when user is trying to enter a new data into the table).
- Assuming there is a number of keys that should be inserted into T table (hash table). Those keys are 10, 02, 26, and 19. T Table has only 7 entries (0 – 6).

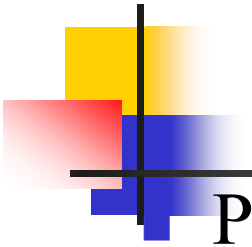
Hash function used is: H(K) = K mod 7

- Address for key “10” => 10 mod 7 = 3
- Address for key “02” => 02 mod 7 = 2
- Address for key “26” => 26 mod 7 = 5
- Address for key “19” => 19 mod 7 = 5



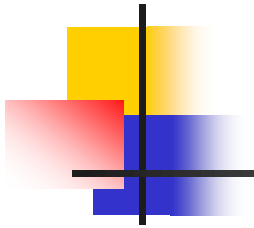


- Searching table using hashing concept should overcome these two problems:
  - ✓ Number of collision(s): Hashing should give minimal number of collisions
  - ✓ Collision problem: Hashing should overcome the problem.



Policies for overcoming collision problem:

- *Linear Probing* – when data cannot be stored in the home address, we resolve the collision by adding 1 to the current address. For example (previous slide), given that we already inserted key "26" to address 005. Next, key "19" is suppose to be inserted in the same address (005) but this address is filled by key "26". Therefore we need to add 1 to the current address (005). At this time, key "19" will be inserted into a new address -> 006. If address 006 is filled by another key, we need to add 1 to the current address (006), and becomes 007. If we have accessed the final address location, addressing will be started at the beginning of the table again.
- Chaining Policy – put the collided key into the same address by extending the location using linked-list.
- Double Hashing @ Rehash – those keys that involve with collision will have to hash continuously until an empty location is found.

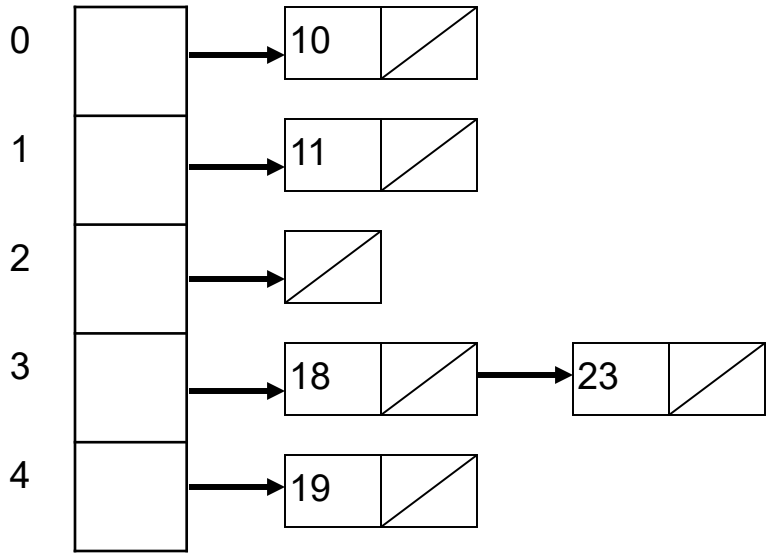


- Example:

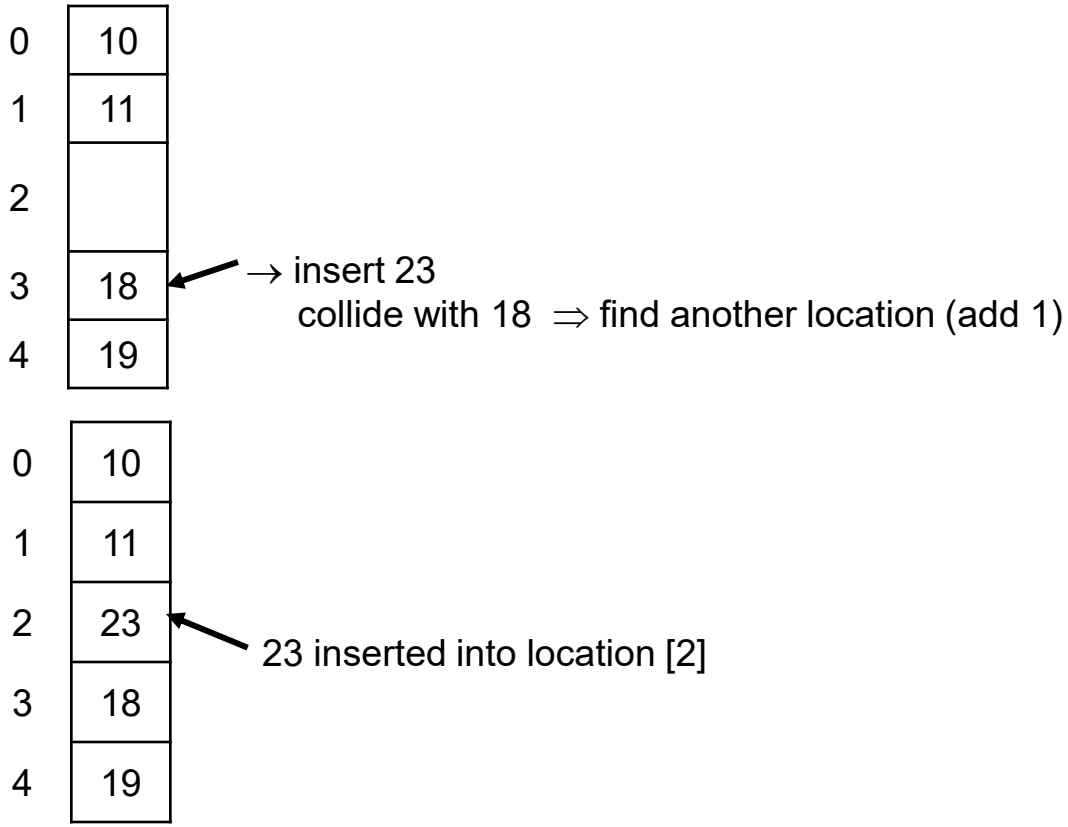
  Given a hash table with 5 locations and hash function that has been used is H(i) = i % 5. Show how this function works if the entries for hash table are 10, 11, 18, 19, and 23 in sequence.

  - Address for key “10” => 10 mod 5 = 0
  - Address for key “11” => 11 mod 5 = 1
  - Address for key “18” => 18 mod 5 = 3
  - Address for key “19” => 19 mod 5 = 4
  - Address for key “23” => 23 mod 5 = 3

a) Using “Chaining” policy



b) Using "Linear-Probe" policy



Double Hashing @ Rehash

- those keys that involve with collision will have to hash continuously until an empty location is found.
- Double hashing uses the idea of applying a second hash function to the key when a collision occurs.
- The result of the second hash function will be the number of positions from the point of collision to insert.
- There are some requirements for the second function:
  - it must never evaluate to zero
  - must make sure that all cells can be probed

- A popular 2nd hash function is:

$$Hash_2(\ key\ ) = R - (\ key \bmod R\ )$$

*Notes: where R is a **first prime number smaller than the size** of the table*

Example 1 – Double Hashing

Table size = 10 elements

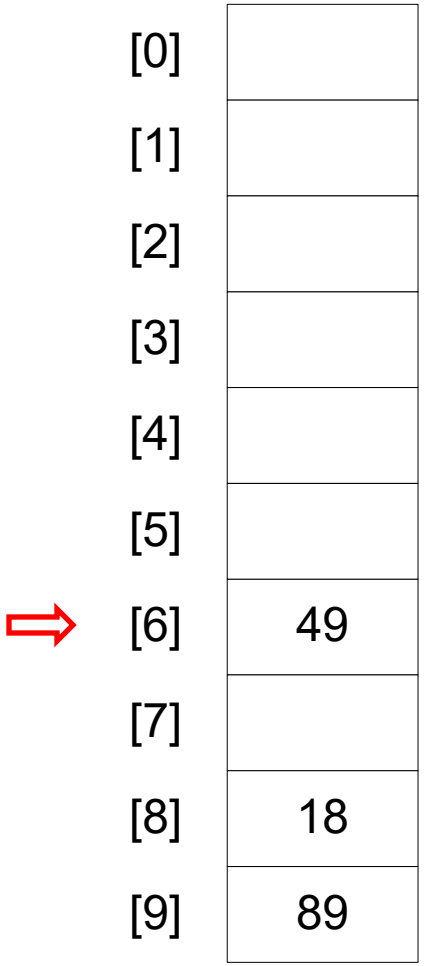
$Hash_1(key) = key \% 10$
$Hash_2(key) = 7 - (key \% 7)$

Insert Keys: 89, 18, 49, 58, 69

HashKey(89) = 89 % 10 = 9

HashKey(18) = 18 % 10 = 8

HashKey(49) = 49 % 10 = 9 a collision!
= (7 – (49 % 7))
= (7 – (0))
= 7 positions from [9]

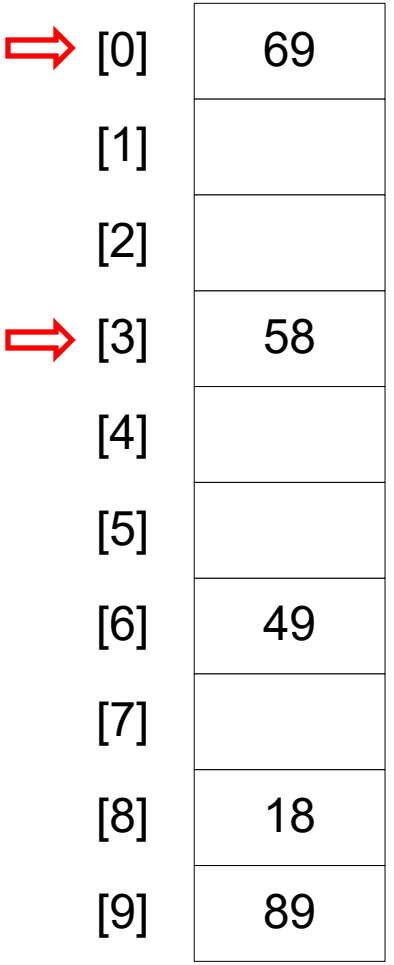


Example 1 – Double Hashing

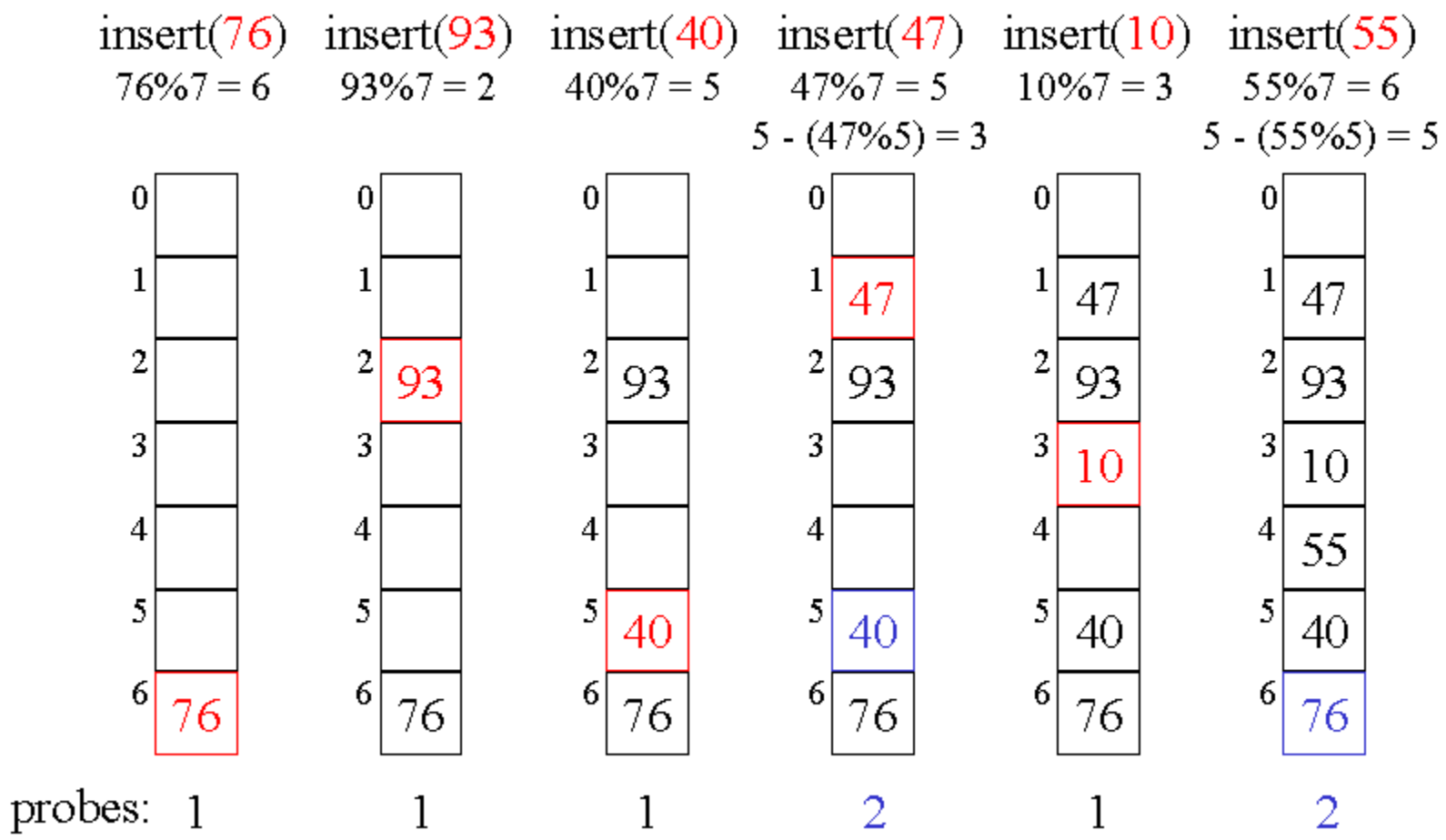
Insert Keys: 58, 69

HashKey(58) = 58 % 10 = 8 a collision!
= (7 – (58 % 7))
= (7 – (2))
= 5 positions from [8]

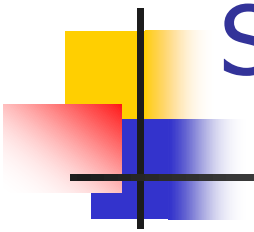
HashKey(69) = 69 % 10 = 9 a collision!
= (7 – (69 % 7))
= (7 – (6))
= 1 position from [9]



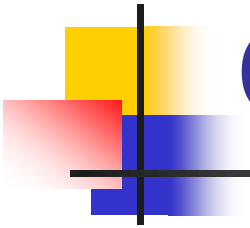
Example 2- Double Hashing



Some Applications of Hash Tables



- **Database systems**: Hash tables are an important part of efficient random access because they provide a way to locate data in a constant amount of time.
- **Symbol tables**: The tables used by compilers to maintain information about symbols from a program. Compilers access information about symbols frequently. Therefore, it is important that symbol tables be implemented very efficiently.
- **Data dictionaries**: Data structures that support adding, deleting, and searching for data. Using a hash table is particularly efficient.
- **Network processing algorithms**: Hash tables are fundamental components of several network processing algorithms and applications, including route lookup, packet classification, and network monitoring.
- **Browser Caches**: Hash tables are used to implement browser caches.



Conclusions

- Hashing is a search method, used when
  - sorting is not needed
  - access time is the primary concern
- Factors affecting efficiency in hash table
  - Choice of hash function
  - Collision resolution strategy
  - Load Factor
- Hashing offers excellent performance for insertion and retrieval of data.

Exercise:

- Using the hash table with eleven locations and the hashing function h(i)=i%11, show the hash table that results when the following integers are inserted in the order given:

26, 42, 5, 44, 92, 59, 40, 36, 12, 60, 80

Assume that the collisions are resolved using

i) Linear probing

ii) Chaining policy

iii) Double hashing with the following secondary hashing function:

$$h_2(i) = \begin{cases} 2i \,\%\, 11 & if\ this\ is\ nonzero \\ 1 & otherwise \end{cases}$$